dSABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Computers

Sanjiang Li *

***Abstract*—A distributed quantum computer (DQC) runs a circuit across several small cores linked by entanglement. Executing a gate whose operands sit in different cores consumes Einstein–Podolsky–Rosen (EPR) pairs, orders of magnitude costlier than a local gate and so the quantity a distributed router must minimise. On a single chip that role falls to SABRE, Qiskit's default layout and routing method, [TODO: This is too weak: Sabre is the SOTA of monolithic quantum transpiler.] which scores a SWAP by the front-layer distance it saves plus a bounded lookahead. We present dSABRE, which carries that objective across cores under a single rule: a move costs the resource it spends, plus the capacity it consumes, plus the progress it buys in one *routing potential*—front-layer distance plus a lookahead discounted by depth in the circuit's directed acyclic graph (DAG). Its added terms are not independent heuristics but the two respects in which a teleport is unlike a SWAP: it fires only at a communication port, and it consumes a free slot in the core it lands in. Across 21 Munich Quantum Toolkit (MQT) Bench circuits at 25–64 logical qubits dSABRE cuts geometric-mean EPR consumption by 49–62% against TeleSABRE, routes three circuits on which TeleSABRE does not converge, and scales to 360 logical qubits on twenty cores; ablation attributes more of the gap to the capacity term than to any other component. Code and online appendices are at https://github.com/ebony72/dsabre.**

***Index Terms*—distributed quantum computing, circuit routing, qubit teleportation, SABRE, EPR minimisation, qubit mapping, lookahead heuristic**

I. Introduction

The near-term path to fault-tolerant quantum computing requires processors with thousands of high-fidelity qubits. No single monolithic chip can economically deliver this scale today. Instead, the field is converging on *distributed quantum computers* (DQC) [1]: multiple small quantum processing units (QPUs, also called cores) interconnected by photonic or microwave links that support remote entanglement generation.

Executing a quantum circuit on a DQC requires a *distributed compilation* pipeline that, like its monolithic counterpart, decomposes into three problem classes [2]. *(i) Qubit allocation* statically assigns logical qubits to cores, typically by hypergraph partitioning [3], [4]. *(ii) Communication-primitive selection* decides how each non-local two-qubit gate is realised—by bringing both qubits to the same core, or by executing the gate remotely [5], [6] (Section II-B describes both primitives and their costs). *(iii) Routing and scheduling* orders the resulting intra-core SWAPs, teleports, and entanglement-generation requests in time, subject to finite entanglement rates and to the capacity of the *communication ports*—the physical qubits that terminate an inter-core link, of which each core has finitely many [7], [8]. All three are judged in one currency: every primitive consumes EPR pairs, whose generation is slow, noisy and rate-limited in current hardware [9], [10]—orders of magnitude costlier than a local gate—so EPR consumption is the dominant objective of distributed compilation, and the one this paper minimises.

Scope. dSABRE addresses the routing component of stage (iii) and leaves scheduling to downstream passes; it is complementary to stage (i), any allocator of which can supply the layout it starts from. Within stage (ii) it uses state teleportation alone. That restriction keeps its accounting unconditional—a teledata pair is consumed by the teleport that creates it, so no candidate score models how long a link stays entangled or how many pairs a core holds at once, and every count we report holds under any entanglement lifetime—while leaving dSABRE measured against a baseline with strictly more primitives.

Two families. Compilers for this stack differ less in their optimisation machinery than in what they assume the machine can do. The first reduces distribution to *(hyper)graph partitioning*, cutting a hypergraph whose hyperedges collect the gates that can share one entangled pair so that the cut weight is the *e-bit* count—one e-bit being one shared EPR pair [3], [4]. The appeal is a global, one-shot optimisation backed by mature solvers; the cost is an idealised machine, in which intra-core routing is free, the number of links joining two cores goes unrecorded, per-core communication memory is effectively unbounded, and an entangled pair survives its whole hyperedge. None of this is incidental to the counts, as our last contribution quantifies.

dSABRE and TeleSABRE belong to the second family, which gives up the global view and decides during routing, pricing those same constraints into each candidate move instead of idealising them away. Routing runs inside a compiler on circuits of the size a DQC exists to execute—this paper routes up to 360 logical qubits on 500 physical ones—so the rule must be a fast per-candidate heuristic. On a single chip that role is filled by SABRE [11], which Qiskit [12] selects by default for both layout and routing and which later work refines rather than replaces [13]: score a move by the change it induces in the distance between the operands of the front-layer gates, plus a lookahead over a bounded window of upcoming gates. The distributed routers closest to this work are built from that objective [14], [15], and the question this paper answers is what it must become when one chip becomes several.

The strongest of them is TeleSABRE [15], which estimates inter-core routing cost by a Dijkstra computation on a con-

S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).
*ORCID: 0000-0002-3332-2546.

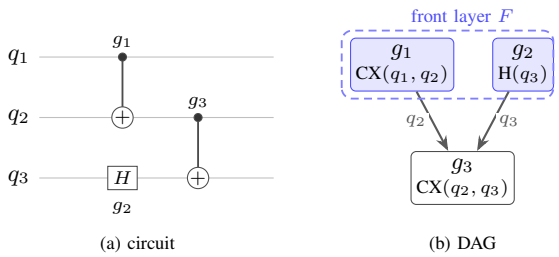
tracted communication graph. Its one substantive limitation is that congestion is handled defensively: a fixed penalty once a core is already nearly full, and a safety valve that drops most of the lookahead once routing stalls, rather than a signal that grades continuously with how much slack a landing core has left, so dense or large instances either burn additional EPR pairs or fail to converge altogether.

The defect is not specific to one penalty: porting SABRE's SWAP-based router to a DQC is not a drop-in substitution of teleport for SWAP. A SWAP is symmetric and leaves the chip's occupied-slot count unchanged; a teleport is one-directional, fires only at a communication port, and is ready only after qubit movement within the source core and eviction of whatever occupies that port at both ends—and once it lands, the destination core is more crowded than before. Scoring it as a bare hop is myopic to all of this; Section III-A scores the whole macro-action instead.

One score, two departures. Our answer is a single scoring rule, not a collection of fixes. Let the *routing potential* Φ be the summed operand distance over the front layer plus a lookahead sum discounting each gate by its DAG depth; Φ is minimised exactly when every front-layer gate is executable, and SABRE's SWAP score is, up to normalisation, the change a move induces in it. dSABRE scores an inter-core move a by $s(a) = \mathrm{cost}(a) + \mathrm{pen}(a) + \Delta\Phi(a)$—resource spent, capacity consumed, progress bought (Section III-A). What the distributed setting adds is then not a list of heuristics but the two respects in which a teleport is unlike a SWAP, one term each. *Availability*: a teleport fires only at a communication port, so a candidate must carry, and be charged for, the SWAPs that stage its qubit there. *Capacity*: it consumes a free slot in the core it lands in, so the score is graded in that core's remaining slack—the one place where the architecture's finiteness enters an otherwise distance-based relaxation. Every component of Section III is one of these two terms, the construction that makes Φ's depth discount well-defined (Section III-E), or a mechanism for the one state in which Φ stops being a usable surrogate—the cores an action needs are saturated (Section III-F). The evaluation divides on the same line, pricing the former in EPR pairs and the latter in completions.

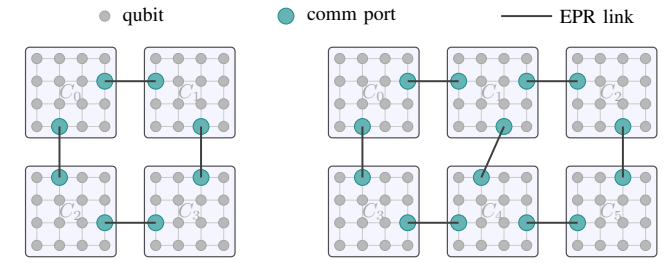
Contributions.

- **A unified, gate-centric teleportation score.** LightSABRE [13] scores a SWAP as the front-layer distance change Δ_F plus a decay-weighted lookahead $\tilde{\Delta}_E$—the change in operand distance over the front gate the move touches, and the depth-discounted sum of that change over a bounded window of upcoming gates—both in Δ-form, new minus old, so lower is better. Together they are $\Delta\Phi$. dSABRE carries both into the inter-core scorer and adds one term per departure: a staging cost d_{prep} for availability and a *core-capacity penalty* c_{cap} for capacity (Section III-D). The capacity term carries the score: removing it inflates geometric-mean EPR by $+80.3\%$ at 25 qubits and, at 64, aborts the three densest circuits outright and more than doubles EPR on the six that still complete ($+101.2\%$), against $+11.3$–31.1% for the lookahead and under 16% for any other single change on those two suites (Section IV-D).
- **BFS-layer inter-core extended set.** The lookahead set is built layer by layer over the remaining DAG, each gate's BFS depth driving the decay $\gamma^{\mathrm{dep}(g)}$. TeleSABRE also slices in layers but weights the g-th emitted gate by $(1 + g/10)$, growing with emission index instead of decaying with depth; only layer expansion fills the size-L window in depth order (Section III-E). Substituting a topological-order extended set costs 15.4% geometric-mean EPR at 64 qubits and is within noise at 25 (-1.4%), the gain growing with circuit size (Section IV-D).
- **A checkpoint–rollback escape with a provable completion bound.** After L_{deadlock} stalled iterations the router restores the last checkpoint and retires the most-stuck gate as one atomic transaction, relaying room along the shortest core-path and teleporting an operand there; either it resolves within D EPR pairs, D the core-graph diameter, or the attempt rolls back at zero cost and the other operand is tried (Section III-F), so termination is unconditional. It fires twice across every table in this paper—64-qubit QPEexact and Random on the heavy-hex star, one of the three instances on which TeleSABRE does not converge at all (the others being the 64-qubit Random circuit and the 360-qubit QFT on twenty cores)—and the graded capacity term and reserved-slot initial layout (Section III-G) keep the router out of these states almost everywhere else in the suite.
- **Comprehensive empirical validation.** Across Munich Quantum Toolkit (MQT) Bench suites at 25, 36, and 64 qubits, dSABRE reduces geometric-mean EPR consumption by 49–62% over TeleSABRE [15], the one baseline that shares its cost model, physical architecture and intra-core SWAP accounting. The margin survives leaving the grid at both levels of the architecture—59% on a ring of IBM heavy-hex cores, 34% on a star—and most of it survives an identical initial allocation on both routers (-38.1% at 25 qubits, -27.7% at 64, against -50.8% and -48.7% from dSABRE's own layout; Sections IV-A and IV-C).
- **A cross-model comparison that states what an EPR pair buys.** `pytket-dqc` [4] distributes by hypergraph partitioning and gate teleportation, so its e-bit counts are *unit-consistent* with ours—the same count of EPR pairs—but not *work-consistent*: an EPR pair does not buy the same amount of routed circuit under each model, so equal counts are not equal work. We correct what can be corrected and measure what cannot: rebuilding its network to this device's own link and slot counts moves the three suite margins to -44.4%, 0.0% and -1.2%; generating the circuits shows 13 of the 21 corrected distributions exceed the communication capacity the device provides; and under a finite entanglement lifetime the narrow 64-qubit margin widens to a 46.5% dSABRE win once a pair may serve only three gates (Section IV-E). Teledata counts need no such correction.



(a) circuit (b) DAG

Fig. 1. DAG for a three-gate circuit on qubits q_1, q_2, q_3. $\mathrm{CX}(q_1, q_2)$ and $\mathrm{H}(q_3)$ share no qubit and have no incoming edges, so both appear in the initial front layer $F = \{g_1, g_2\}$ and may execute in parallel. $\mathrm{CX}(q_2, q_3)$ depends on both via qubits q_2 and q_3; it enters F only after g_1 and g_2 have been executed. Circuit depth is 2 (two sequential layers).



(a) 2×2 B-grid
4 cores, 64 qubits, 4 EPR links

(b) 2×3 H-grid
6 cores, 96 qubits, 7 EPR links

Fig. 2. DQC architectures used in this paper (B-grid and H-grid families introduced by TeleSABRE [15]). Small grey circles are qubits; teal circles are inter-core communication ports; bold lines are EPR-channel links between cores. Each core is a 4×4 grid. (a) 2×2 B-grid. (b) 2×3 H-grid.

II. Background

A. Quantum Circuits

In this paper, a quantum circuit acts on n *qubits* through a sequence of *gates*: *one-qubit (1Q) gates* (Hadamard, rotations, ...) and *two-qubit (2Q) gates*, of which the *CX* (CNOT) gate is the standard primitive. The two classes are together universal—CX and the single-qubit unitaries decompose every n-qubit unitary exactly [16]—and a finite basis such as $\{\mathrm{CX}, H, T\}$ suffices approximately; it is what the benchmark circuits of Section IV-A are transpiled into. The *SWAP* gate exchanges the states of two adjacent qubits; it decomposes into three CX gates and is what routing algorithms insert to migrate logical qubit states across the hardware.

A circuit is represented as a *directed acyclic graph* (DAG) in which each node is a gate and each directed edge records a data dependency (qubit produced by gate g_1 consumed by gate g_2). For every qubit q, each gate acting on q is connected to the *next* gate acting on q: the second cannot begin until the first has written its output on q. Figure 1 shows a small example, where *circuit depth* is the length of the longest path through the DAG.

B. Quantum Teleportation

Quantum teleportation [5] transfers the state of a qubit from a sender to a receiver—in the DQC setting, from one core to another over an inter-core link—using a pre-shared *EPR pair* (a maximally entangled two-qubit state) and two classical bits. The source qubit's state is destroyed at the sender and reconstructed at the receiver; no physical qubit travels across the link. Each EPR pair is consumed upon use, so the *EPR count* is the primary inter-core routing cost in distributed quantum compilation. Throughout this paper, "EPR" as a unit (e.g. "10 EPRs") denotes EPR-pair count; "EPR pair" is used when emphasising the physical entangled state itself.

Two inter-core teleportation variants arise in DQC compilation. *Teledata* moves a logical qubit to a new core (one EPR pair consumed), after which the qubit participates in local gates there. *Telegate* [6] executes a non-local 2Q gate directly across two cores via a shared EPR pair, without physically relocating either qubit. The two price a circuit differently. Teledata pays once per relocation and then makes *every* subsequent gate on that qubit local, so its cost amortises over the qubit's residence in a core; telegate pays once per non-local gate, or once per group of gates when consecutive non-local gates on a shared control are amortised onto a single entangled state. Which is cheaper is circuit-dependent: amortisation rewards structured circuits with shared controls, relocation rewards qubits with long runs of local work. The two also make different demands on the hardware. A teledata pair is consumed by the teleport that creates it, whereas an amortised telegate needs its link to stay entangled—and to occupy a link qubit—for the span of the gate group it serves, so gate-teleportation counts are contingent on entanglement lifetime and on how many pairs a core can hold at once. Both assumptions are load-bearing for gate-teleportation compilers, and Section IV-E measures what they are worth on one of them.

C. Distributed Quantum Architecture

We model a DQC architecture $\mathcal{A}$ abstractly as a weighted graph $G_{\mathcal{A}} = (V, E, w)$ whose vertices are physical qubits. The vertex set is partitioned into K *cores* $V = V_1 \sqcup \cdots \sqcup V_K$; intra-core edges E_{intra} describe in-core connectivity and carry weight 1, and inter-core edges E_{inter} connect designated *communication ports* on neighbouring cores and carry a much larger weight $w_{\mathrm{link}} \gg 1$ reflecting the cost of an EPR pair. A communication port is a physical qubit like any other and may also hold a logical (data) qubit between teleportations; the "communication" label only indicates that the port is one end of an inter-core EPR link, not that the slot is reserved. dSABRE treats $G_{\mathcal{A}}$ as a black box and only queries it through three distance tables: physical d_{phys}, intra-core d_{intra}, and core-graph d_{core}. The router therefore extends to *any* DQC topology — arbitrary intra-core graphs, mixed core sizes, sparse inter-core meshes, hierarchical trees of clusters, etc. — by simply supplying these tables.

For experimental simplicity our reference implementation instantiates $G_{\mathcal{A}}$ as a *grid-of-grids* [15]: each core is an $m \times m$ 2D nearest-neighbour grid, and the cores themselves form an $r \times s$ super-grid joined by a small set of inter-core links between specific boundary qubits. Figure 2 shows two concrete topologies used in our experiments.

a) Hierarchical distance precomputation.: Because intra-core and inter-core costs decouple, the all-pairs distance

d_{phys} over $G_{\mathcal{A}}$ admits a two-level shortest-path computation that is substantially cheaper than running Dijkstra on the full $|V|$-vertex graph. First, all-pairs distances are computed once *inside* a single core V_i (size $|V_i| = M$) by breadth-first search in $O(M^2)$ time, giving d_{intra}. Second, the *core graph* (a multigraph of K super-nodes connected by inter-core links) is solved in $O(K^2)$ time, giving d_{core}. Third, a port-to-port table d_{port} is obtained by Dijkstra over the $O(K)$ communication ports alone—two ports of one core joined at their d_{intra} distance, the ends of each inter-core link at the link weight—in $O(K^2 \log K)$ time. Every $p \to q$ path leaves p's core through some port and enters q's through another, so for $p \in V_i$, $q \in V_j$ with Π_i the ports of core i,

$$d_{\text{phys}}(p,q) = \min_{\pi \in \Pi_i,\ \pi' \in \Pi_j} d_{\text{intra}}(p,\pi) + d_{\text{port}}(\pi,\pi') + d_{\text{intra}}(\pi',q), \tag{1}$$

taking $d_{\text{intra}}(p,q)$ as well when $i{=}j$ so a route that leaves and re-enters one core is never missed. Equation 1 is exact, not an approximation, and is verified against a dense all-pairs table on every pair of every architecture used in this paper. This brings precomputation from $O((KM)^2 \log(KM))$ down to $O(KM^2 + K^2 \log K)$ and resident space from $\Theta(P^2)$ to $O(KM^2 + K^2)$, and is the only architecture-dependent setup dSABRE performs; a d_{phys} query then costs $O(|\Pi|^2)$ and is memoised, so routing sees $O(1)$ lookups after the first.

D. Qubit Routing and SABRE

Let C be a circuit DAG on n *logical qubits* $\mathrm{q}_1, \ldots, \mathrm{q}_n$, to be run on an architecture with $P = |V| \geq n$ physical qubits. An *initial layout* is an injection $\ell : \mathrm{q}_i \mapsto \mathrm{p}_{\sigma(i)}$ placing them on distinct physical slots, where σ is a permutation on $[P] = \{0, 1, \ldots, P-1\}$. A 2Q gate can execute only when its two operands occupy adjacent vertices of the *coupling graph* ($G_{\mathcal{A}}$ in the distributed case), so *routing* inserts SWAP and teleportation operations—as few as possible—to make every gate executable in turn. Cost model: each local SWAP costs c_{swap}; each inter-core teleport costs c_{tele} (one EPR pair).

SABRE [11] is the reference heuristic for this problem on a single chip, and the one dSABRE builds on: Qiskit [12] selects it by default for both layout and routing, and later work has refined its scoring form, on single chips and in the multi-core setting [14], [15] alike. LightSABRE [13], the refinement Qiskit ships, selects the SWAP minimising:

$$H = \frac{1}{|F|} \sum_{g \in F} \Delta_F(g) + w \frac{1}{|E|} \sum_{g \in E} \Delta_E(g), \tag{2}$$

where F is the front layer, E the extended lookahead set, and $\Delta(g) = d^g_{\text{new}} - d^g_{\text{old}}$ the change in physical distance between the two qubits of gate g caused by the candidate SWAP (negative when the SWAP moves them closer); d^g is shortest-path distance on the coupling graph. All extended-set gates contribute with equal weight; the $\frac{1}{|E|}$ factor normalises the lookahead term to the same scale as the front term. SABRE also couples its router with an initial-layout optimiser: alternating forward and reverse-circuit passes, each seeded with the previous pass's final layout, anneal towards a layout self-consistent with the circuit's gate structure.

III. dSABRE

dSABRE is a SABRE-style routing algorithm for multi-core processors. Of the two communication primitives of Section II-B it uses teledata alone, for the accounting reason given in the introduction; adding telegate candidates to the scorer is a genuine extension and we scope it as future work (Section VI). The complication is not the primitive itself but what makes it pay off: a telegate candidate is cheap only if its link stays entangled across several non-local gates, which turns scoring into a *grouping* decision—which gates to bundle onto one pair—rather than the single-candidate $\Delta\Phi$ of Eq. 7. `pytket-dqc` makes that decision globally via hypergraph partitioning (Section IV-E); folding an online analogue into a greedy score without losing teledata's unconditional accounting is the open problem.

dSABRE accepts *any* legal initial layout—any injective assignment of logical qubits to physical slots that respects per-core capacity. We propose a SABRE-inspired layout with a per-core capacity reservation (Section III-G), which is used as the default initial layout of dSABRE.

The core invariants and notation are: (i) the layout $\ell : \mathrm{q}_i \mapsto \mathrm{p}_{\sigma(i)}$ of Section II-D assigns logical to physical qubits and the inverse ℓ^{-1} records which logical qubit (if any) currently occupies each physical slot; (ii) the working DAG W holds the remaining gates (initially the full circuit) and shrinks as gates are executed; (iii) the front layer F is the set of gates in W with no unexecuted predecessors, and the extended set E is a fixed-size sliding lookahead beyond F. Table I collects every symbol used in this section.

A. Scoring: Potential, Cost, and Capacity

dSABRE moves qubits with two different actions—an intra-core SWAP and an inter-core teleport—and chooses between them by a priority rule rather than a common score: whenever a front-layer gate has both operands in one core, the router applies one intra-core SWAP (Section III-C); only when no such gate remains does it score teleport candidates (Section III-D). The two scorers thus run at different scopes—per-core and taint-restricted for SWAPs, global for teleports—and are never compared against one another.

The intra-core scorer is SABRE restricted to a core, with the depth discount of Eq. 5. The teleport scorer needs more care. An inter-core teleport differs from an intra-core SWAP in two ways, and each difference adds one term to the score.

(i) Availability: a SWAP is legal on every edge of the coupling graph, whereas a teleport can fire only at a communication port. A candidate must therefore also carry the intra-core SWAPs that bring its qubit to that port, and any eviction needed to free the port itself. We treat that whole sequence as a single *macro-action* and score it as one, so the router always compares complete moves, each priced by what carrying it out actually costs. *(ii) Capacity*: a SWAP exchanges the contents of two slots and leaves the set of occupied slots unchanged, so capacity never binds on a monolithic chip; a teleport instead moves a qubit into a different core and consumes one of that core's free slots, so a candidate must be charged for the capacity it takes.

Scoring complete moves is what keeps the choice from being short-sighted. Were the teleport the bare hop alone, the SWAPs that walk a qubit to a port would be chosen earlier by a rule that does not know which teleport they will enable, and an occupied port would delete the move rather than make it dearer. Bundling the preparation into the action instead lets one arg min weigh the full cost of reaching *every* neighbouring core against the progress each would buy. Section III-D adds one term for each of (i)–(ii), on top of the front-plus-lookahead distance objective inherited from SABRE; the rest of this subsection fixes that objective.

Fix a layout ℓ and let $d(g;\ell)$ be the weighted graph distance between the two operands of a 2Q gate g on the full-chip graph $G_{\mathcal{A}}$ (intra-core edges weight 1, inter-core links weight w_{link}). Two distinct qubits are never at distance 0, so $d(g;\ell) = 1$, its minimum, exactly when g is executable; every other value is the SWAP-equivalent hops still separating its operands. Define the *routing potential*

$$\Phi(\ell) \;=\; \sum_{g\in F} d(g;\ell) \;+\; w_e \sum_{g\in E} \gamma^{\text{dep}(g)}\, d(g;\ell), \qquad (3)$$

the first term measuring work that must be done now, the second discounted work that is coming, with $\gamma^{\text{dep}(g)}$ discounting a gate by its depth because a deeper gate is more likely to be displaced before it is reached. Φ is a surrogate, not the true cost-to-go: its front term is exact at the boundary ($\sum_{g\in F} d(g;\ell) = |F|$ iff every gate in F is executable), and elsewhere the standard SABRE relaxation, which ignores that qubits contend for the same slots. Φ itself is never evaluated in absolute terms, only the difference $\Delta\Phi$ below, and an additive shift in d cancels there—so, unlike Eq. 6's per-core H_{intra}, which normalises by $|F_c|$ and $|E_c|$ to compare candidates across cores of different active-gate counts, Eq. 3 carries no denominator: there is no shared cardinality across the single state it is evaluated at.

A teleport a, together with the staging SWAPs that precede it, transforms ℓ into ℓ_a. Write $\Delta\Phi(a) = \Phi(\ell_a) - \Phi(\ell)$, let $\text{cost}(a)$ be the resource the action consumes, and let $\text{pen}(a)$ charge the capacity constraint that Φ omits. dSABRE greedily minimises

$$s(a) \;=\; \underbrace{\text{cost}(a)}_{\text{resource spent}} \;+\; \underbrace{\text{pen}(a)}_{\text{capacity}} \;+\; \underbrace{\Delta\Phi(a)}_{\text{progress bought}} \;. \qquad (4)$$

Section III-D instantiates each: $\text{cost}(a)$ is the staging count d_{prep}, $\text{pen}(a)$ is the capacity penalty c_{cap}, and $\Delta\Phi(a) = \Delta_F + w_e\tilde{\Delta}_E$ is the induced change in the two sums of Eq. 3, restricted to the gates on the teleported qubit, which are the only gates the move affects. The sums are left unnormalised: the candidate set spans cores, so no per-core cardinality is shared across candidates, and the size of E is instead bounded by the cap $|E|_{\max}$. Within one iteration every candidate spends exactly one EPR pair, so the teleport's own EPR cost is constant across the arg min and does not appear in Eq. 4; $\text{cost}(a)$ carries only the staging SWAPs, which do vary between candidates.

Two properties of this score are what the experiments later test. First, the capacity term is not a tie-breaker added to a distance heuristic; it is the one place where the architecture's finiteness enters an otherwise pure relaxation, and it is *graded* in the landing core's free-slot count rather than switching on at a fixed occupancy. Section IV-D finds it the largest single contributor to dSABRE's EPR advantage, ahead of the lookahead term and well ahead of every other component. Second, the lookahead enters only through $\gamma^{\text{dep}(g)}$, so a gate's influence decays with its DAG depth; Section III-E shows that this discount is well-defined only if E is built in BFS layers.

The mechanism of Section III-F exists because a greedy minimiser of Eq. 4 can reach states where Φ stops being a usable surrogate—every cheap action blocked by saturated cores, or no action reducing $|W|$ at all. We report its value in terms of feasibility and completion rather than as EPR reductions.

TABLE I
NOTATION FOR SECTION III, WITH THE DEFAULT VALUE OF EACH HYPERPARAMETER. DEFAULTS ARE USED UNCHANGED IN EVERY EXPERIMENT AND ON EVERY ARCHITECTURE.

Symbol	Meaning	Default
Routing state		
$\ell,\ \ell^{-1}$	layout $\mathrm{q}_i \mapsto \mathrm{p}_{\sigma(i)}$, and its inverse	—
W	working DAG of unexecuted gates	—
$F,\ E$	front layer; extended (lookahead) set	—
$F_c,\ E_c$	their restrictions to core c	—
$K,\ M$	number of cores; qubits per core	—
$n,\ P$	logical qubits in C; physical qubits $\lvert V\rvert{=}KM$	—
Distances and scores		
$G_{\mathcal{A}}$	full-chip graph (Section II-C)	—
$d(g;\ell)$	weighted distance between operands of g	—
$d_{\text{intra}},\ d_{\text{core}}$	intra-core and core-graph distance	—
$\text{dep}(g)$	depth of g (BFS layer, inter-core set)	—
Φ	routing potential, Eq. 3	—
Δ_F	change in d over the front gate	—
$\tilde{\Delta}_E$	decay-weighted change over E, Eq. 5	—
d_{prep}	staging + eviction SWAPs before a teleport	—
c_{cap}	capacity penalty on the landing core	—
f_{nxt}	free slots in the landing core c_{next}	—
$\pi_s,\ \pi_d$	source/destination communication ports	—
n_s	staging slot: neighbour of π_s nearest p_1	—
Hyperparameters		
c_{swap}	SWAP cost	3
c_{tele}	teleport (EPR) cost	10
τ	capacity threshold	3
c_{pen}	capacity penalty multiplier	15
w_{link}	inter-core link weight	10
w_e	extended-set weight	0.25
L	extended-set capacity	20
γ	lookahead decay	0.9
L_{deadlock}	stalled iterations before rollback	50
$N^{\max}_{\text{backup}}$	maximum rollbacks	50

B. Workflow

Figure 3 sketches the main routing loop; the corresponding pseudocode is in the online appendices. Each iteration runs four phases: **(P1)** drain F (execute 1Q gates and any 2Q gate whose operands are already adjacent in the same core, iterated to a fixed point); **(P2)** partition the remaining front into F_{intra} (both operands in one core) and F_{inter} (operands in different cores); **(P3)** if $F_{\text{intra}} \neq \emptyset$ apply one intra-core SWAP (Section III-C), else score teleportation candidates over F_{inter}

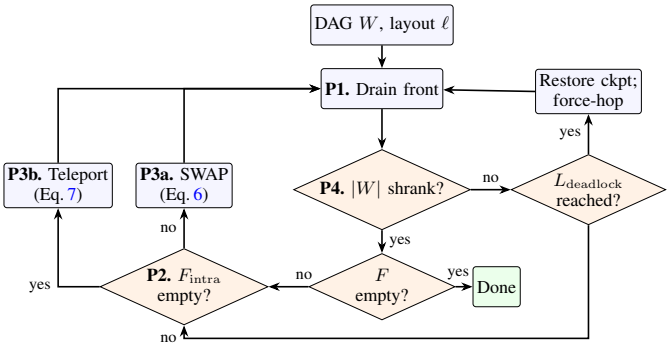


Fig. 3. The dSABRE routing workflow. Each iteration drains the front layer—executing 1Q gates and adjacent intra-core 2Q gates (P1)—classifies the remaining front into intra-core and inter-core gates (P2), and applies either the best intra-core SWAP (P3a, $\arg\min H_{\text{intra}}$) or the best gate-driven teleport candidate (P3b, $\arg\min s$). It checkpoints when $|W|$ falls (P4); after L_{deadlock} stalled iterations it instead restores the checkpoint and force-hops the stuck gate.

and apply the cheapest (Section III-D); **(P4)** checkpoint (ℓ, W) if $|W|$ decreased, otherwise trigger checkpoint–rollback after L_{deadlock} stalled iterations and abort after $N^{\max}_{\text{backup}}$ failed recoveries.

C. Intra-Core SWAP Heuristic

When the front contains a gate whose operands share a core but are not adjacent, dSABRE chooses an intra-core SWAP using the standard SABRE heuristic, restricted to the relevant core. Let $F_c = \{g \in F_{\text{intra}} : \text{core}(g) = c\}$ be the front gates in core c and $E_c = (g_0, g_1, \ldots)$ the corresponding intra-core extended set, an *ordered list* of at most L gates (default L=20) collected in topological order. The traversal stops including a gate the moment either of its qubits has previously appeared in a cross-core gate in the working DAG—a qubit is marked once it is found ineligible and the mark propagates to every later gate on it, the same rule dataflow analysis calls *taint propagation*—ensuring E_c contains only gates that remain local to core c. Each gate g_i is assigned a depth $\text{dep}(g_i)$, the number of intra-core gates on the longest qubit-wire path from the front layer to g_i (front layer = depth 0; first successor = depth 1). The lookahead contribution is

$$\tilde{\Delta}^c_E = \sum_{i=0}^{|E_c|-1} \gamma^{\text{dep}(g_i)} (d^{g_i}_{\text{new}} - d^{g_i}_{\text{old}}), \tag{5}$$

where $\gamma \in (0, 1]$ is the lookahead decay and the distance terms use the intra-core graph. The tilde marks the departure from SABRE's flat extended-set weighting, for the reason given after Eq. 3. Section III-E explains how $\gamma^{\text{dep}(g)}$ interacts with the BFS-layer extended set, where $\text{dep}(g)$ is the BFS depth rather than an iteration index. For every neighbour pair (u, v) within a core, the score

$$H_{\text{intra}} = \frac{\Delta^c_F}{|F_c|} + w_e \frac{\tilde{\Delta}^c_E}{|E_c|} \tag{6}$$

is the SABRE objective for intra-core scoring with the lookahead-decay weighting of Eq. 5. The lowest-score SWAP is applied; tie-breaking is by enumeration order. Because the scorer is restricted to a single core and the candidate set is the $O(M)$ intra-core neighbour pairs, the per-core selection is trivially parallel across active cores.

D. Inter-Core Teleportation Scoring

Operational definition. Executing a candidate is deterministic given the state at the moment it is scored. The destination port π_d need not be empty: an occupant is evicted to the nearest free slot in c_{next} by the same rule applied to the source port π_s, and only cores with a free slot are ever proposed as c_{next} to begin with, so this eviction cannot fail. The one exception—a fully-occupied *source* core, whose correction can itself consume the destination's reserved slot and so must be re-checked—and the resulting update to ℓ and ℓ^{-1} are given in full in the online appendices.

For each gate $g \in F_{\text{inter}}$ on logical qubits q_1, q_2, let c_{src} and c_{tgt} be the cores currently hosting q_1 and q_2 respectively (i.e. $\ell(q_1) \in c_{\text{src}}$ and $\ell(q_2) \in c_{\text{tgt}}$, where ℓ is the current layout). Both endpoints are tried as teleportation sources. For the qubit chosen as source (say q_1 at physical position $p_1 = \ell(q_1)$), the router enumerates every neighbouring core c_{next} and every inter-core link (π_s, π_d) between c_{src} and c_{next}. Each such triple $(q_1, c_{\text{next}}, (\pi_s, \pi_d))$ is one *candidate*; Figure 4 illustrates the structure. Executing a candidate involves two steps, plus eviction of either comm port if occupied (operational definition above): (1) intra-core SWAPs that move q_1 from p_1 to the *staging slot* n_s, the neighbour of π_s closest to p_1; and (2) a teleportation that consumes the EPR pair on link (π_s, π_d) and delivers q_1 from n_s to π_d in c_{next}. Both steps belong to the action and are scored together, so a teleport is available from *anywhere* in the source core and an occupied port raises a candidate's score rather than removing it (Section V contrasts this with TeleSABRE). Each candidate is scored as:

$$s = \underbrace{d_{\text{prep}}}_{\text{staging}} + \underbrace{c_{\text{cap}}}_{\text{capacity}} + \underbrace{\Delta_F + w_e\, \tilde{\Delta}_E}_{\Delta\Phi \text{ (front + lookahead)}}, \tag{7}$$

where lower scores are preferred. The last group is SABRE's front-plus-lookahead objective (Eq. 2), the whole score on a monolithic chip; the first two are the availability and capacity terms of Section III-A. Each term is defined as follows.

Staging cost d_{prep} is the number of intra-core SWAPs needed to move q_1 from p_1 to staging slot n_s, plus eviction SWAPs to free the communication ports if occupied:

$$d_{\text{prep}} = d_{\text{intra}}(p_1,\, n_s) + d_{\text{evict}}(\pi_s) + d_{\text{evict}}(\pi_d).$$

Capacity penalty $c_{\text{cap}} = c_{\text{pen}} \max(0, \tau - f_{\text{nxt}})$ discourages routing into nearly-full cores. f_{nxt} is the number of free slots in the immediate landing core c_{next} (not the final partner-qubit core c_{tgt}), τ the capacity threshold, and c_{pen} a cost multiplier. This discourages progressive crowding of high-demand cores and empirically reduces the cascading evictions associated with saturated landing cores.

Front-layer change $\Delta_F = d^g_{\text{new}} - d^g_{\text{old}}$ is the change in weighted physical distance between q_1 and q_2 after the teleport (negative when they are brought closer), computed on the full-chip graph $G_{\mathcal{A}}$ (intra-core edges weight 1, inter-core

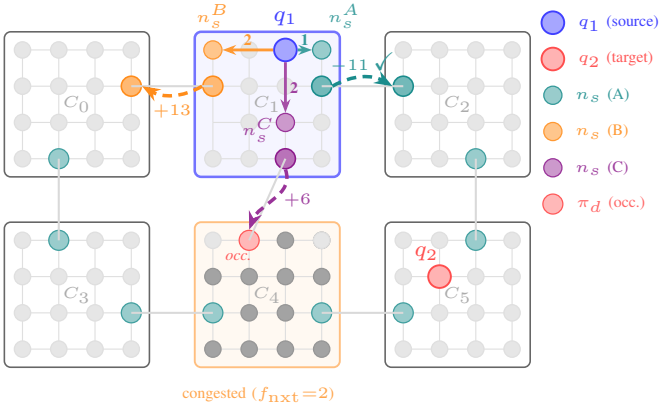


Fig. 4. How dSABRE chooses where to teleport a qubit. A single cross-core gate $g=(q_1, q_2)$ is pending, with q_1 in core C_1 (blue border) and its partner q_2 in C_5; the router scores the three links leaving C_1 and takes the cheapest by Eq. 7. Executing any one of them takes two steps: intra-core SWAPs first move q_1 to the staging slot n_s (count on the solid arrow), then a teleport along the link (π_s, π_d) carries it into the next core (dashed arc, annotated with the resulting score s). The three candidates isolate one decision factor each, and correspond row by row to Table II. **A** (teal) moves towards C_5 and wins, $s_A=-11$. **B** (orange) moves to C_0, away from C_5, so its front-layer term turns positive. **C** (violet) makes the same progress as A but lands in the nearly-full core C_4 (orange border, two free slots), where the capacity penalty outweighs the progress. The red port marked *occ.* is already occupied, so C also pays an extra eviction SWAP.

TABLE II
SCORE BREAKDOWN FOR THE THREE CANDIDATES OF THE RUNNING EXAMPLE (FIGURE 4). LOWER IS BETTER; CANDIDATE A IS SELECTED.

Candidate	next core	direction	d_{prep}	c_{cap}	Δ_F	score s
A	C_2	towards C_5	1	0	−12	**−11**
B	C_0	away from C_5	2	0	+11	13
C	C_4	towards C_5	3	15	−12	6

links weight w_{link}). Exactly one front-layer gate involves q_1: gates sharing a qubit are DAG-ordered, so at most one can be unblocked at a time, and g is already that gate by the premise of this scoring step.

Decay-weighted lookahead. $\tilde{\Delta}_E$ is Eq. 5 with two adaptations for the inter-core scorer: E is the inter-core extended set (Section III-E) rather than the per-core taint-propagated list, and the sum is restricted to gates involving the teleported qubit, the only ones the move affects.

a) Running example.: Figure 4 and Table II score the three outgoing-link candidates on the 2×3 H-grid at defaults, isolating one factor each, and show the three groups of Eq. 4 deciding between them: A wins on the cheapest staging plus the largest progress; B moves away from C_5, turning Δ_F against it; C buys A's progress but lands in a nearly-full core, where the capacity term alone reverses the decision.

E. Inter-Core Extended-Set Construction

The lookahead $\tilde{\Delta}_E$ in Eq. 7 depends on how the inter-core extended set E is built. SABRE [11] fills E in topological order, interleaving all wires uniformly; TeleSABRE [15] uses BFS layers but within each layer emits gates in arbitrary order and weights each by its iteration index g via $(1 + g/10)$ rather than by depth. Both schemes can deprioritise the very follow-on gates a candidate teleport would help, causing $\tilde{\Delta}_E$ to undercount its benefit.

dSABRE keeps a BFS-layer expansion of the remaining DAG with two refinements: (i) within each layer, gates sharing a qubit with the front are emitted first; (ii) every gate in BFS layer k is assigned $\text{dep}(g)=k$, used in the decay $\gamma^{\text{dep}(g)}$, so the size-L truncation cuts cleanly at a layer boundary and weighting decays exponentially with depth rather than growing linearly with iteration index. It adds no hyperparameter and runs in $O(|E|_{\max})$ time per call when remaining in-degrees are maintained incrementally: BFS peels zero-in-degree gates layer by layer, each emitted gate triggering $O(1)$ decrements on its successors. The empirical contribution over the topological variant is reported in Section IV-D.

The construction matters structurally. In Eq. 3 a gate's influence is set by $\gamma^{\text{dep}(g)}$, so the lookahead is meaningful only if the gates surviving the size-L truncation are the shallow ones—and BFS layering delivers that by construction, emitting layer $k+1$ only once layer k is exhausted, so every gate of depth $\leq k$ enters E before any gate of depth $>k$. Topological filling has no such property, at any L: on a DAG whose first wire carries a chain of L gates and whose second carries a lone depth-1 gate h, a topological sweep may spend the entire budget on the chain and omit h—the very gate that becomes executable one layer ahead, and so the one whose distance term a teleport decision most needs. This orders the window, which is not by itself a bound on EPR count, but it is why the gain from BFS layering grows with circuit size (Section IV-D): the deeper the remaining DAG, the further a topological sweep drifts from the front before L is spent.

F. Feasibility Mechanism: Checkpoint–Rollback

One mechanism sits outside the score of Eq. 4, answering the state in which Φ has stopped being a usable surrogate because the cores an action needs are saturated: every cheap action blocked, or no action reducing $|W|$ at all.

Checkpoint–rollback. LightSABRE's [13] release valve forces shortest-path progress on the most-stuck front gate; we make the forcing step an atomic, checked transaction with a checkpoint around it. Every iteration that reduces $|W|$ saves (ℓ, W), and after L_{deadlock} stalled iterations the router restores that state, then relays room to each core along $\text{core\_path}(c_1, c_2)$ and teleports one operand there hop by hop; a failed attempt rolls back at zero cost and the other operand is tried, and whichever succeeds retires the gate immediately. The router aborts after $N_{\text{backup}}^{\max}$ failed recoveries. The transaction buys the progress guarantee the greedy rule lacks: relaying room needs only that the chip's total free capacity exceed its core count—a quantity every teleport and SWAP conserves—so a successful attempt unites the gate's operands within D EPR pairs, D the core-graph diameter, after which $|W|$ strictly decreases. Termination is unconditional; only *completion* rests on that capacity condition, a second reason the reserved slots of Section III-G matter.

Checkpoint–rollback prices feasibility, not EPR pairs, and fires twice across every table in this paper—64-qubit QPEexact and Random on the heavy-hex star, both single activations re-

solved immediately. Almost nothing else in the current evaluation saturates enough cores to need it. We report its value as a worst-case guarantee accordingly—termination unconditional, completion bounded whenever the capacity condition above holds—not as an EPR reduction.

G. Initial Layout and Complexity

Both the capacity penalty and the checkpoint–rollback escape depend on the initial layout leaving free slots; this is how it is produced. dSABRE bootstraps from Qiskit `SabreLayout` [11] run on the full architecture graph $G_{\mathcal{A}}$ (intra-core edges plus inter-core links), with two adaptations for the distributed setting.

Per-core corner reservation. SabreLayout packs each core to capacity, which both stalls the inter-core scorer—an occupied port cannot accept a teleport without first evicting its occupant—and trips the capacity penalty on every candidate landing there. We therefore withhold from SabreLayout's coupling map the k most-remote *corner* qubits of *every* core: the minimum-degree vertices farthest, by total shortest-path distance, from all others, which on both our architectures sit furthest from any inter-core link. Each core thus keeps k free slots and stays below the capacity threshold. We take k adaptively, the largest value in $\{0,\dots,4\}$ holding the usable-slot fill $n_q/(N_{\text{cores}}(m^2-k))$ at or below 0.80: $k{=}4$ on the 25/36/100/200-qubit suites, $k{=}2$ on the denser 64-qubit one. Reserving *per core* is what matters—under core-major numbering a whole-chip rule reserves slots in one core and packs every other. Since the rule reads only vertex degree and remoteness it transfers to any core shape, selecting leaf qubits on the heavy-hex cores of Section IV-B where it selects corners on a grid.

Forward–backward–forward schedule. We run three passes—forward, then the reversed DAG, then forward again—and keep the better of the two forward passes by EPR. The backward pass starts from the first pass's output layout and yields one that pre-positions qubits near their early interaction partners, from which the third warm-starts. This is SABRE's backward refinement [11], and it outperforms pure-forward warm-restart. Best of three SabreLayout seeds is reported.

Cost. Write $N{=}|C|$ for the gate count, N_r for the gates still unrouted, D_M and D_K for the intra-core and core-graph diameters, and A, B, T for the intra-core SWAPs, the teleports, and the iterations the router performs; the rest is as in Table I. Distance tables are precomputed once per architecture in $O(KM^2{+}K^2\log K)$ (Section II-C) and are not charged per circuit.

Scoring is charged per *applied move* rather than per iteration. An iteration applies one SWAP in each core that holds an unexecuted local gate, so pricing a whole-chip per-iteration cost against an iteration count derived from one move per iteration would count the same per-core parallelism twice. The number of (iteration, active core) pairs is exactly A; every iteration applies at least one move, so $T=O(A+B)$; and a gate needs $O(D_M)$ SWAPs and $O(D_K)$ teleports, one teleport crossing a whole core, so $A=O(ND_M)$ and $B=O(ND_K)$. Scoring one intra-core SWAP costs $O(|F_{\text{c}}|{+}L)$: front-layer gates are qubit-disjoint, so a candidate exchanges the contents of two slots and can move at most *two* of them, leaving every other term of Δ_F identically zero. Scoring one teleport costs $O(Mn{+}P)$: $O(n)$ candidates at $O(M{+}L)$ each—the M is the scan for an eviction target—over an $O(P)$ free-slot census. Both lookahead sets are cheap once in-degrees are carried between calls: $O(n{+}L)$ for the inter-core set, and $O(L{+}M)$ per core for E_{c}, which unfolds to the forward closure of F_{c} through gates whose every 2Q ancestor is intra-core—a Kahn expansion seeded at F_{c} and pruned at each wire's first non-local gate emits it without reading the rest of the DAG. Hence

$$T_{\text{total}} = O\Big(\underbrace{N(D_M{+}D_K)(n{+}L{+}M)}_{\text{lookahead}} + \underbrace{ND_M(M{+}L)}_{\text{SWAP scoring}} + \underbrace{ND_K(Mn{+}P)}_{\text{teleport scoring}}\Big) \tag{8}$$

under the iteration budget, past which the router aborts. Charging per move rather than per iteration removes a factor M from the SWAP term; $D_M{+}D_K$ is $O(\sqrt{M}{+}\sqrt{K})$ on a grid-of-grids but linear in K on a core ring; and the bound is linear in N. The router we measure is not—compile time $N^{2.1}$—because it builds E_{c} by a sweep of the whole topological order instead, at $\Theta(N_r)$ a call. The full derivation, what the seeded closure changes, and a measured validation are in the online appendices. Restricting intra-core scoring to core-local edges reduces the front-layer term by up to a factor K over flat SABRE [11] on the same P qubits, when front gates spread across cores.

IV. Experimental Evaluation

A. Setup

Benchmarks. We use three circuit sets from the MQT Bench suite [17], all in the IBM native-gate set (Qiskit opt3 transpilation, measurements and barriers removed).

Suite 1 (25-qubit): quantum amplitude estimation (AE), quantum Fourier transform (QFT), quantum neural network (QNN), random circuit (Random), GHZ state preparation, and graph-state preparation.

Suite 2 (36-qubit): six circuits spanning a broader range of algorithmic families and CX densities: Bernstein–Vazirani (BV, linear chain), Deutsch–Jozsa (DJ, oracle), W-state preparation (cascaded star), VQE with SU(2) ansatz (layered variational), QAOA (dense all-to-all), and quantum phase estimation (QPEexact, hierarchical).

Suite 3 (64-qubit): the same six circuit families as Suite 1 scaled to 64 logical qubits (AE, QFT, QNN, Random, GHZ, Graphstate), run on the larger H-grid architecture, extended with three further circuits that complete the algorithmic-class coverage at the largest scale: QPEexact (phase estimation, the class on which gate teleportation is strongest), QAOA (dense max-cut cost layers with shared-control ZZ structure), and a 16-bit Multiplier (arithmetic, the suite's largest circuit at 13 040 CX). Per-circuit properties (qubits, CX count, depth, CX per qubit) and circuit provenance are tabulated in the online appendices.

Architecture. Suites 1 and 2 use a 2×2 B-grid: four 4×4 cores (64 physical qubits, 4 EPR links). Suite 3 uses a 2×3 H-grid: six 4×4 cores (96 physical qubits, 7 EPR links). See Figure 2 for the full qubit-level topology. The 25q suite on the B-grid and the 64q suite on the H-grid follow the benchmark setup of TeleSABRE [15] (we add a 36q B-grid suite to broaden circuit diversity). Section IV-C0a adds two further architectures, built from IBM heavy-hex cores wired as a ring and as a star, to test whether the router's advantage depends on the grid-of-grids structure above.

Baselines. TeleSABRE [15] is the baseline every headline number is measured against, because it shares dSABRE's cost model, physical architecture, and intra-core SWAP accounting. It is also a SABRE-style router that jointly optimises intra-core SWAPs, teledata (qubit moves), and telegate (remote CX execution) operations, run using its compiled C binary with the `optimize_initial` flag enabled (built-in layout heuristic + 3-success sampling) on the same device.

We also compare against `pytket-dqc` [4], a Quantinuum library that partitions a circuit's interaction hypergraph with KaHyPar and then *embeds* each hyperedge as a sequence of gate teleportations sharing one EPR pair—a Steiner tree or cover embedding spanning the servers the hyperedge touches—so multiple non-local gates on the same wire amortise onto a single pair. The machine-model caveats listed in Section I, all of them in `pytket-dqc`'s favour, are large enough that the comparison needs a section of its own (Section IV-E).

What one EPR pair buys. All three tools report in the same unit—one EPR pair consumed is one e-bit charged, "EPR" and "e-bit" used interchangeably throughout (Section I)—but what each unit *buys* differs, and no cross-tool comparison is meaningful without stating the difference. dSABRE uses *teledata*: one EPR per inter-core hop relocates a logical qubit, after which subsequent intra-core gates on it are free. TeleSABRE uses that primitive and telegate, and we count each operation once. `pytket-dqc`, not a router but a static partitioner, amortises many gates onto one pair under an unbounded-entanglement-lifetime assumption (Section II-B). Its column is therefore unit-consistent with the others but not work-consistent, in the sense given in the Introduction; the online appendices tabulate what a pair buys under each tool's model.

Parameters. Table I lists all hyperparameters; the defaults are used unchanged on every suite and architecture. Layout passes: 2 (forward–backward–forward). Each router uses its authors' convention: best-of-3 seeds for dSABRE and TeleSABRE(vacuously for the latter, see below) [11], [15], best-of-5 for `pytket-dqc` [4], which reflects operational cost rather than per-attempt variance. TeleSABRE, however, proves deterministic under `optimize_initial` with its Hungarian layout—on all eight 64-qubit circuits it completes, the three seeds return identical counts—so the whole benefit of that convention accrues to dSABRE. The honest way to bound it is to strip it: on those same eight circuits, replacing best with median over the three `SabreLayout` seeds moves the gmean gap from -48.7% to -42.0%.

B. Main Results

TeleSABRE [15] supports three operation types: intra-core SWAPs, teledata (qubit movement, 1 EPR), and telegate (remote CX execution via the cat-entangler protocol, 1 EPR). We report TeleSABRE's EPR count as the sum of teledata and telegate operations, so each EPR pair is counted exactly once. Unless stated otherwise, dSABRE is run with its default initial-layout configuration (Section III-G). Subsequent tables and ablations inherit this default. We report two metrics in the main tables: EPR pairs and intra-core SWAPs. This subsection carries the headline TeleSABRE comparison over the three benchmark suites and the large-QFT points; Section IV-C then repeats it on architectures that are not grids at either level, pushes it to 360 logical qubits, and asks whether it survives a shared initial layout and a harsher cost model.

25q. dSABRE reduces geometric-mean EPRs over TeleSABRE by **50.8%** (Table III). Per-circuit reductions range from -11.1% on QNN to -84.6% on Graphstate.

36q. dSABRE reduces geometric-mean EPRs by **61.6%** (Table III), with per-circuit reductions of -25% to -83% and no circuit at parity. The simple structured circuits BV, VQE-SU2, and W-state are essentially solved (1 EPR on BV; 6–10 EPRs on the others). dSABRE also uses fewer intra-core SWAPs than TeleSABRE on every circuit of this suite, 79 against 173 in geometric mean.

64q. The H-grid suite is where the gap from TeleSABRE is most consequential (Table III). The suite comprises the six TeleSABRE-benchmark circuits plus QPEexact, QAOA, Multiplier. dSABRE reduces gmean EPRs by **48.7%** over the eight circuits on which TeleSABRE converges, with per-circuit reductions ranging from -4.9% on the 13 040-CX Multiplier to -74.2% on Graphstate, and QFT/QNN at -38.5%/-44.1%. The margin is narrower here than on the B-grids, and narrowest on the largest circuit. On Random, TeleSABRE fails to converge on any of three seeds; dSABRE completes it (712 EPR) under default scoring alone, without triggering checkpoint–rollback.

C. Generality and Scale

a) Architecture independence: heavy-hex cores on a ring and on a star: Section II-C models the architecture as an abstract weighted graph queried only through three distance tables, but everything above runs on a grid of grids. If the scoring or the capacity mechanism tacitly assumed grid structure at either level, that generality claim would be empty, so we vary both levels at once. The core becomes a 27-qubit IBM heavy-hex tile of the Falcon family [18], which shares none of the grid's regularity: degrees range over $\{1,2,3\}$, six qubits are degree-1 leaves, and the diameter is 12 against 6 for the 4×4 grid. Four such tiles (108 physical qubits) are wired two ways—a *ring* (4 links, every core of degree 2) and a *star* (3 links, core 0 a hub through which every inter-core route must pass). The star is deliberately the harsher of the two, and is the case in which a scorer that quietly assumed a homogeneous core graph would break. Nothing is specialised for either topology: the corner reservation of Section III-G is stated over vertex degree and remoteness, so it selects leaf

TABLE III

MAIN RESULTS: DSABRE AGAINST TELESABRE, THE ONE BASELINE THAT SHARES ITS COST MODEL, PHYSICAL ARCHITECTURE, AND INTRA-CORE SWAP ACCOUNTING. EPR = TELEDATA + TELEGATE FOR TELESABRE, TELEDATA FOR DSABRE; SWAP COUNTS INTRA-CORE SWAPS. NEGATIVE ΔEPR FAVOURS DSABRE. TELESABRE IS BEST-OF-3 SEEDS, DSABRE BEST-OF-3 SABRELAYOUT SEEDS UNDER THE FWD→BWD→FWD SCHEDULE. *TELESABRE FAILS TO CONVERGE; THE GEOMETRIC MEAN IS TAKEN OVER THE CIRCUITS IT COMPLETES, AND WHERE THAT IS NOT THE WHOLE SUITE A SECOND ROW GIVES DSABRE'S MEAN OVER EVERY CIRCUIT IT ROUTED.

		TeleSABRE		dSABRE		
Circuit	CX	EPR	SWAP	EPR	SWAP	ΔEPR (%)
25 logical qubits, B-grid 2×2 of 4×4 cores (64 physical)						
ae	558	26	292	22	253	−15.4
ghz	24	4	36	1	4	−75.0
graphstate	25	13	47	2	15	−84.6
qft	580	46	405	33	332	−28.3
qnn	1223	54	659	46	649	−14.8
random	1124	271	1248	181	1208	−33.2
gmean (6)		31.1	234	15.1	126	**−51.3**
36 logical qubits, same B-grid						
bv	17	6	39	1	13	−83.3
dj	35	4	63	3	23	−25.0
qaoa	1200	268	1165	134	994	−50.0
qpeexact	1019	190	883	62	574	−67.4
vqe_su2	105	28	134	9	45	−67.9
wstate	70	13	79	6	40	−53.8
gmean (6)		27.6	173	10.5	82	**−62.0**
64 logical qubits, H-grid 2×3 of 4×4 cores (96 physical)						
ae	1962	353	1494	200	1547	−43.3
ghz	63	16	132	6	39	−62.5
graphstate	64	62	281	15	88	−75.8
qft	1966	312	1711	176	1397	−43.6
qnn	8126	889	5471	495	5344	−44.3
random*	1627	—	—	718	3621	—
qpeexact	2139	289	1706	184	1565	−36.3
qaoa	3920	956	4189	538	3759	−43.7
multiplier	13040	1601	9654	1285	8969	−19.7
gmean (8)		284.5	1564	145.4	1097	**−48.9**
gmean (9)		—	—	173.6	1252	—
QFT scalability, 5×5 cores throughout, core count 6 → 12 → 20						
100q	3420	248	2414	236	2585	−4.8
200q	7220	1232	7089	825	7166	−33.0
360q*	13300	—	—	2168	15885	—

qubits where it previously selected grid corners, and reads k=4 per core off the 59% fill on its own.

dSABRE keeps its advantage on both core graphs. On the ring both routers complete all six circuits and the margin is **−59.0**%; read on the five circuits TeleSABRE completes on both architectures it is −62.3% against the H-grid's −57.9%, so leaving the grid widens the gap rather than costing it, driven by QNN, where TeleSABRE needs 1289 EPR against dSABRE's 238. On the star it *narrows* to −34.0% over the five circuits TeleSABRE completes, and one circuit reverses outright: on AE dSABRE uses 144 EPR against 132 — the only loss dSABRE takes anywhere in this paper. TeleSABRE fails on Random, which dSABRE routes at 481 EPR. Per-circuit counts for both topologies are in the online appendices.

b) Large circuits: To stress-test dSABRE beyond the main evaluation range we run it and TeleSABRE on the MQT-Bench QFT at **100**, **200** and **360** logical qubits. Core size is held at 5×5 throughout and only the core count grows—2×3 (6 cores, 150 physical), 3×4 (12, 300), 4×5 (20, 500)—so logical qubits per core stay near-constant at 16.7, 16.7 and 18.0 while the core graph lengthens, diameter 3 → 5 → 7. Holding occupancy fixed is what makes the three rows one series rather than three unrelated points.

Table III reports the three points. dSABRE leads at both sizes where TeleSABRE converges (−3.6% at 100q, −37.7% at 200q), and at **360q on twenty cores TeleSABRE does not converge on any of three seeds**—it returns without success in 537–645 s—while dSABRE completes at 2208 EPR (one of three `SabreLayout` seeds aborts; the reported count is the best of the other two). The margin widens with diameter: at 100q's diameter-3 core graph there is little for the capacity term and lookahead to decide, so the margin is small (−3.6%); at 200q's diameter-5 graph, where longer inter-core paths give those mechanisms real decisions to make, the margin grows substantially (−37.7%).

A companion experiment varying core granularity at fixed circuit finds the headroom a wider core buys and the diameter a shorter core graph costs close to a wash for dSABRE at this operating point (online appendices).

c) Is the margin an artefact of the setup?: Two things could make the headline gap something other than a routing result: a more favourable initial layout on dSABRE's side, and a cost model that prices EPR pairs generously. Controlling for the first—re-running dSABRE from TeleSABRE's own `optimize_initial` layout, so only the routing loop differs—the gap persists at −38.1% on 25q and −27.7% on 64q, against −50.8% and −48.7% from dSABRE's own layout, putting most of the margin in the routing loop—three-quarters at 25q, a little over half at 64q. Sweeping c_{tele} from 10 to 100 at fixed c_{swap}=3 (online appendices) controls for the second and only widens the gap: dSABRE saves 33.5–57.1% at the teleport-friendly end and 41.5–60.6% at the photonic-link end [1], [9], [10], improving monotonically in c_{tele} on every suite.

d) Compile time: The online appendices tabulate wall-clock compilation time for all three compilers on the 64-qubit suite. In geometric mean TeleSABRE leads at 3.2 s, dSABRE follows at 19.0 s, and `pytket-dqc` trails at 65 s, nearly flat in circuit size because the partitioner sets it. Between the two routers the gap is partly super-linear: the E_c sweep of Section III-G makes dSABRE quadratic in gate count (measured $N^{2.1}$), so the ratio drifts about one order of magnitude across a 200× range of circuit size—13× slower on the 13 040-CX Multiplier, 10× *faster* on Graphstate, where TeleSABRE's sampling needs many iterations to converge. The rest is per-operation overhead against compiled C++ with cached lookups. Restructuring the E_c scan, not a change of language, removes the super-linear part—10.5× on this suite, measured in the online appendices; EPR pairs, not compile seconds, are the scarce resource on this hardware.

TABLE IV

ABLATION OF THE THREE SCORING TERMS THAT PRICE EPR PAIRS. ONE COMPONENT IS DISABLED PER ROW; ALL OTHERS REMAIN AT THEIR DEFAULTS, AND THE PROTOCOL AND HARDWARE BUDGET ARE THE MAIN RESULTS'. EACH SUITE IS ABLATED OVER ITS *whole* CIRCUIT SET—SIX AT 25Q, NINE AT 64Q—SO THE **FULL** BASELINES ARE TABLE III'S OWN GEOMETRIC MEANS (15.1 AND 173.6) RATHER THAN A SUBSET'S. THE REMAINING MECHANISM, CHECKPOINT–ROLLBACK, DOES NOT PRICE EPR PAIRS AND IS DISCUSSED IN SECTION III-F INSTEAD. †TWO OF NINE CIRCUITS (QNN, MULTIPLIER) ABORT OUTRIGHT WITHOUT THE CAPACITY PENALTY; THE GMEPR OF THE SEVEN THAT COMPLETE IS 207.8 AGAINST THE FULL BASELINE'S 112.3 OVER THE SAME SEVEN (+85.1%), AND IS NOT COMPARABLE TO THE NINE-CIRCUIT BASELINE.

	25q		64q	
Configuration	gmEPR	Δ (%)	gmEPR	Δ (%)
Full (baseline)	15.1	—	173.6	—
No capacity penalty ($c_{\text{pen}} = 0$)	25.7	+69.6	207.8†	+85.1†
No extended-set lookahead ($w_e = 0$)	17.1	+12.6	228.4	+31.5
Topological extended set (not BFS)	15.2	0.0	206.0	+18.7

TABLE V

`PYTKET-DQC` AGAINST DSABRE, BY SUITE. *published* IS THE NETWORK IT IS NORMALLY GIVEN, *physical* THE ONE MATCHING DSABRE'S DEVICE, WHICH IS THE COLUMN READ IN THE TEXT. SUITE FIGURES ARE GEOMETRIC MEANS OVER THE CIRCUITS OF TABLE III. PER-CIRCUIT COUNTS ARE IN THE ONLINE APPENDICES.

	`pytket-dqc` e-bits		dSABRE		
Suite	published	physical	EPR	Δ (%)	unreal.
25 logical qubits	35.3	27.5	15.1	−45.1	4/6
36 logical qubits	12.3	10.6	10.5	− 0.9	2/6
64 logical qubits	168.0	176.3	173.6	− 1.5	7/9

D. Ablation Study

We ablate dSABRE's design choices along three axes: the scoring terms that price EPR pairs, the mechanisms that instead price feasibility, and continuous parameter sensitivity. All configurations inherit the defaults of Table I and report the *geometric-mean EPR count* (gmEPR) over the whole of each suite, so every figure below shares Table III's basis.

a) Mechanism ablation.: Table IV disables one component at a time on the 25q and 64q suites, by setting the corresponding parameter to zero or substituting the topological-order extended-set construction for the BFS-layer one. The two suites stress different regimes: the 25q B-grid is sparsely loaded (qubits per core ≤ 7, ample headroom on every core), so congestion-driven mechanisms barely activate, while the 64q H-grid runs at ∼67% qubit utilisation and saturates inter-core ports under dense workloads.

The capacity penalty is the single most critical component on both suites. At 25q, disabling the penalty costs EPR count—+80.3% gmean. At 64q **the three densest circuits (QAOA, QNN, Multiplier) abort outright** regardless of iteration budget, and the six that still complete pay for it in cost rather than failure: gmean EPR more than doubles (+101.2%), from parity on GHZ to +348.2% on AE. This is the empirical core of the paper's design claim: the one term that encodes the architecture's finiteness is not merely the one that matters most, it is the one whose absence the router survives only by paying double, and on its densest instances does not survive at all. The extended-set lookahead is second, contributing +11.3% on 25q and +31.1% on 64q; removing it collapses the scorer to a near-myopic policy, and its value grows with circuit size as dependency chains lengthen —QAOA 517 → 848, Multiplier 1523 → 2123. Substituting a topological-order extended-set construction costs 15.4% on 64q; at 25q the two constructions are within noise of each other (−1.4%), where the DAG is too shallow for the difference to register. The gain still grows with the depth of the remaining DAG, as Section III-E predicts.

b) Parameter sensitivity.: Sweeping each continuous hyperparameter one-at-a-time on 25q and 64q, every default sits on a flat region of its curve (within ∼2% of the local minimum); only pushing a control far from its default causes a sharp regression, e.g. c_{pen}=0 doubles gmEPR. No single hyperparameter is a hidden lever; the full sweep is in the online appendices.

E. Comparison with `pytket-dqc`

`pytket-dqc` is not a router but a hypergraph partitioner emitting gate teleportations. Of the four differences listed in Section I, two are architectural assumptions rather than cost-model conventions, and both can be corrected; Table V reports the comparison before and after, throughout using its strongest distributor, COVEREMBEDDINGSTEINERDETACHED, best of five seeds.

Correcting the network. The network `pytket-dqc` normally given is more permissive than our device in two respects. Its per-server *communication capacity*—the number of link qubits a core may hold simultaneously, `server_ebit_mem` in the tool—is unbounded unless set, whereas a core here has one port per incident link, $\deg(c) = 2$ or 3; and its server sizes split the *logical* qubit count evenly, whereas a core physically offers $M - \deg(c)$ data slots.

The corrected counts are still not executable. Communication capacity does not reach the cost model: as noted in [19], none of the distribution routines consider it, satisfaction being deferred to a pass over the generated circuit. Declaring it accordingly left all 21 distributions we generated unchanged. Generating the circuit is what tests executability, and usually it fails: **13 of the 21 distributions cannot be built within the communication capacity the device provides**. AE at 25 qubits needs 19 simultaneous link qubits per core against the 2 available. This is no coincidence: a hyperedge over g gates buys a factor of g in e-bits and pays in link-qubit occupancy, so the cheapest-looking distributions demand the most memory. Nor does the repair pass that would restore executability reach these circuits: on the 25-qubit suite it returns only on the two distributions that already fit, and not within 300 s on any of the four that do not.

What survives the corrections. Read with that caveat in force—most `pytket-dqc` counts below are the cost of a distribution its own tool cannot build on this device (Table V, last column)—the direction is set by circuit structure rather than size. dSABRE wins five of six circuits at 25 qubits; at 64

the comparison reverses only on QPEexact, where Steiner-tree embedding reuses a few pairs across many controlled operations (+72.9%)—AE, a clear `pytket-dqc` win in an earlier router generation, is now within a percent of parity (−0.7%)—while dSABRE leads on the densest circuits (Multiplier −47.3%, QNN −17.9%). Allocation quality accounts for part of it: given `pytket-dqc`'s own KaHyPar partition, so that only the routing differs, dSABRE leads on all three suites (−56.8%, −11.9%, −41.5%).

Entanglement lifetime. A further assumption is separable from the last: communication capacity is how many pairs are live at once, lifetime is how long one must live, and the suite separates them—Random exceeds the device's ports while being nearly insensitive to the entanglement-lifetime bound $D_{\max}$ introduced next. Every e-bit count above assumes an *unbounded* lifetime, a link qubit staying entangled until the last gate of its hyperedge. Bandini *et al.* [20] argue this is unrealistic and cap gates-per-EPR at $D_{\max}$, proposing $D_{\max}$=3. Re-scored under that bound by a counter that generalises `pytket-dqc`'s own cost model and reproduces it at $D_{\max}=\infty$—dSABRE needs no adjustment, a teledata pair being consumed instantly—the `pytket-dqc` lead on this per-cell distributor selection (+15.0% at $D_{\max}=\infty$) survives only above a threshold: the geometric-mean gap crosses parity between $D_{\max}=\infty$ and 10 and becomes a dSABRE win of **46.5**% at $D_{\max}$=3. Gate teleportation's e-bit advantage is therefore conditional on sustaining entanglement across $\gtrsim$10 gates per pair; teledata routing delivers its counts under any lifetime. Per-circuit sweeps are in the online appendices.

V. Related Work

The Introduction already places dSABRE within the partition–communicate–schedule decomposition of the DQC compilation stack and lists the major lines of work in each stage. Here we focus on what specifically distinguishes dSABRE from the closest prior art.

SABRE-style distributed routers. TeleSABRE [15] is our closest prior work and direct experimental baseline. Both routers follow the SABRE loop, but differ in how they score inter-core moves: TeleSABRE evaluates candidates via a *contracted communication-graph energy* that averages per-gate Dijkstra distances over the whole front layer, whereas dSABRE uses a local, gate-centric score (Eq. 7) evaluated only on the gates the moving qubit touches (Section III-D).

Three further differences are where the measured gap comes from. *Availability*: TeleSABRE's teleport is a bare hop, proposed only when the qubit already sits beside a free port whose far end is free; dSABRE's is a macro-action priced in d_{prep}, enumerating every neighbouring core and turning occupancy into cost rather than letting it delete the action. *Selection*: TeleSABRE pools SWAP, teleport and telegate candidates in a single arg min and subtracts a flat bonus of 100 from the communication primitives—59.5% of its communication operations at 64 qubits fire while a same-core front gate is still pending, a state in which dSABRE, which inverts the priority, cannot teleport at all. *Capacity*: a fixed penalty on any core with at most two free slots, charged along the whole Dijkstra path, against dSABRE's $c_{\text{pen}}\max(0,\tau - f_{\text{nxt}})$ graded in the landing core's slack—the ablation attributes more to this term than to any other. The online appendices give each difference in full. TeleSABRE supports telegate candidates while dSABRE currently uses teledata only.

DMapS [14] is a concurrent end-to-end DQC compiler pairing a KaHyPar partitioner with a SABRE-derived router, and shares our joint EPR-plus-local-SWAP objective. Its only cross-core primitive is a two-EPR *remote SWAP*, where dSABRE uses one-way teledata into a layout-reserved free slot. dSABRE consumes 2–4× fewer EPRs on the dense 25q circuits and 1.6–1.8× fewer at 64q, while DMapS wins on maximally sparse circuits that KaHyPar partitions almost perfectly; the code-level audit, per-circuit head-to-head and a fill-ratio sweep showing parity near full occupancy are in the online appendices.

Allocation methods built on hypergraph partitioning [3], [4] are complementary to dSABRE: any can supply the initial layout its routing loop consumes, and given `pytket-dqc`'s own partition dSABRE routes it more cheaply than that partitioner's gate teleportations do (Section IV-E). Similarly, time-aware schedulers [7], [8] sit upstream or downstream of the routing stage dSABRE addresses.

VI. Future Work

Composing dSABRE with burst executionand telegate. dSABRE optimises EPR count on a fixed circuit DAGusing teledata alone (Section III); burst execution and telegate are two views of the same next step, not two separate ones. Burst-execution passes such as AutoComm [21] and its collective extension QuComm [22] operate earlier in the toolchain: they use gate commutativity to reorder consecutive non-local gates on the same wire (or across multiple wires) and amortise the whole block onto a single shared entanglement state, reporting EPR savings of 50–75% on amenable circuits; the lifetime-bounded gate grouping of Bandini *et al.* [20] tempers such savings with the $D_{\max}$ constraint we adopt in Section IV-E. That amortisation is exactly what a telegate candidate needs to be worth scoring: Section III identifies the obstacle as a *grouping* decision, and a burst-extraction pre-pass is one way to supply it externally, exposing the wire-aligned sequences `pytket-dqc`'s hypergraph partitioning otherwise finds in one global shot (Section IV-E) for dSABRE's BFS extended set to score as a unit. This is also where the reversal of Section IV-E comes from—gate teleportation beats dSABRE on QPEexact and AE, whose long shared-control sequences a burst pre-pass would expose—so closing that gap and adding telegate are one open question, not two. Quantifying the joint improvement—especially on circuits where a burst's beneficiary core is itself a function of the routing decisions dSABRE makes—remains open. That the interaction is non-trivial is suggested by Babu *et al.* [23], who pair gate teleportation with a SABRE-style mapper on a single heavy-hex IBM Eagle processor and report 10–25% depth reduction over stock SABRE: even intra-chip, teleportation pays off only where the mapper is told to expect it.

Parallel teleportation for circuit-runtime minimisation. dSABRE optimises EPR count and emits one teleport per

iteration; wall-clock runtime depends additionally on whether teleports with disjoint links and staging sets can fire concurrently. A natural follow-on is a scheduling pass over dSABRE's teleport trace that batches non-conflicting moves, co-designed with denser inter-core links [24] and entanglement buffer qubits [22], [25]. Time-sliced compilers [7], [8] provide a starting point but are not paired with a SABRE-style scoring loop; naively adding links without retuning the routing energy can *increase* EPR consumption.

Beyond EPR count: latency and fidelity. This paper optimises a resource count, not an execution time or an output fidelity. Two of the three system-level costs are already partly visible: intra-core SWAP count, the dominant contributor to routed depth, is reported beside EPR count in every table and falls alongside it, and entanglement quality enters through the lifetime bound of Section IV-E, since capping gates-per-EPR is how a decaying link appears in a cost model. What remains open is a joint objective, and it is the same scheduling layer as the parallel-teleportation pass proposed above, asked to do more: EPR count, depth and per-operation error rates are not interchangeable—a teleport is slow and noisy, a SWAP fast and cheap, and the exchange rate is hardware-specific—so a fidelity-aware scheduling pass needs a device error model in addition to the conflict check concurrency alone requires, both outside the routing stage dSABRE addresses. The cost-ratio sweep is a first step: the conclusions are stable across the two orders of magnitude of relative teleport cost that separate microwave-coupled from photonic-link hardware.

VII. Conclusion

We presented dSABRE, a SABRE-style router for multi-core distributed quantum computers. It scores every inter-core move by one rule—the resource the move spends, plus the capacity it consumes, plus the change it induces in a single routing potential—whose two added terms answer the two respects in which a teleport is unlike a SWAP (Section III-A). Within an iteration it resolves intra-core front-layer gates by SWAP scoring and scores teleport candidates only when the intra-core front is empty, both phases sharing a common decay-weighted lookahead form. Two scoring choices account for most of the gap to the state of the art: an explicit capacity penalty that strongly discourages moves into low-slack cores, which contributes more than any other single component, and a BFS-layer inter-core extended set that respects DAG dependencies. Between them, these two carry dSABRE through instances on which the state of the art does not converge at all—the 64-qubit Random circuit, Random on the heavy-hex star, and the 360-qubit QFT on twenty cores—almost entirely without the checkpoint–rollback escape, which backstops both with a provable worst-case bound and fires only twice in this paper (Section III-F). More broadly, the results indicate that distributed routing benefits less from a richer per-candidate scoring formula than from ensuring that the architectural capacity signal reaches the scorer in *graded* form: continuously scaled to how full the landing core already is, the way c_{cap} responds to free-slot count in Eq. 7, rather than acting only as a feasibility filter or a fixed-threshold step, the binary condition Table IV shows costing 80.3% of geometric-mean EPR at 25 qubits when it is removed, and outright non-convergence on the three densest 64-qubit circuits (Section III-A).

Artefact availability.: The dSABRE implementation, the routed-circuit benchmark suites, per-circuit JSON results, and the online appendices referenced throughout this paper are available at https://github.com/ebony72/dsabre.

Acknowledgments

This work was partially supported by the Australian Research Council (Grant No. DP250102952 and DP220102059). The author used Anthropic's Claude (via Claude Code) to assist with: (i) refactoring portions of the dSABRE implementation, (ii) authoring benchmark scripts and aggregating per-circuit JSON results into the tables of Section IV, (iii) drafting the LaTeX sources for the architecture and teleport-candidate figures (Figures 2 and 4), and (iv) copy-editing prose throughout the manuscript. All AI-generated content was reviewed, verified, and corrected by the author, who takes full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] P. Andres-Martinez and C. Heunen, "Automated distribution of quantum circuits via hypergraph partitioning," *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.

[4] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.

[8] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[9] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[10] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[11] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[12] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2024.

[13] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "LightSABRE: A lightweight and enhanced SABRE algorithm," 2024. [Online]. Available: https://arxiv.org/abs/2409.08368

[14] T. Luo, Y. Zheng, Y. Deng, and X. Fu, "DMapS: End-to-end qubit mapping and routing for distributed quantum computing architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025, code: https://github.com/RoccoLoter/DMapS.

[15] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.
[16] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," *Physical Review A*, vol. 52, no. 5, pp. 3457–3467, 1995.
[17] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.
[18] C. Chamberland, G. Zhu, T. J. Yoder, J. B. Hertzberg, and A. W. Cross, "Topological and subsystem codes on low-degree graphs with flag qubits," *Physical Review X*, vol. 10, no. 1, p. 011022, 2020.
[19] P. Andres-Martinez, T. Forrer, D. Mills, J.-Y. Wu, L. Henaut, K. Yamamoto, M. Murao, and R. Duncan, "Distributing circuits over heterogeneous, modular quantum computing network architectures," *Quantum Science and Technology*, vol. 9, 2024, arXiv:2305.14148.
[20] M. Bandini, D. Ferrari, S. Carretta, and M. Amoretti, "Optimized compilation for distributed quantum computing," 2026, arXiv:2602.24062.
[21] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.
[22] A. Wu, Y. Ding, and A. Li, "QuComm: Optimizing collective communication for distributed quantum computing," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.
[23] A. P. Babu, O. Kerppo, A. Muñoz Moller, M. Haghparast, and M. Silveri, "Gate teleportation-assisted routing for quantum algorithms," 2025.
[24] P. Escofet, S. B. Rached, S. Rodrigo, C. G. Almudever, E. Alarcón, and S. Abadal, "Interconnect fabrics for multi-core quantum processors: A context analysis," in *Proceedings of the 16th International Workshop on Network on Chip Architectures (NoCArc)*, 2023, arXiv:2309.07313.
[25] J. Liu, A. Zang, M. Suchara, T. Zhong, and P. D. Hovland, "Hardware-software co-design for distributed quantum computing," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–6.

Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.